

Vellum Discord AI Bot – Roadmap & Smart Architecture

Overview

- This document outlines a practical roadmap and scalable architecture for building the Vellum Discord AI Bot.
- The goal is to create a system that manages the Discord server, analyzes community feedback, and produces
- actionable development insights while remaining efficient and easy to maintain.

4■Stage Development Roadmap

- Stage 1 – Server Automation
 - • Welcome messages
 - • Rule acceptance system
 - • Auto role assignment
 - • Basic spam filtering
 - • Channel guidance
- Stage 2 – Feedback Collector
 - • Monitor channels like #bug-reports, #feature-requests, #support-chat
 - • Store messages in a database (SQLite initially)
 - • Capture metadata such as timestamp, user, and thread
- Stage 3 – AI Summaries
 - • Integrate a local open■source model via Ollama
 - • Classify messages (bug, feature request, confusion, noise)
 - • Extract key information
 - • Generate daily development reports
- Stage 4 – Intelligence System
 - • Cluster duplicate bug reports
 - • Score contributors automatically
 - • Assign contributor roles
 - • Build a dashboard showing issues, summaries, and contributors

Smart AI Pipeline Architecture

- To keep the system efficient, do not send every message to the AI.
- Pipeline:
- Discord Message
- ↓
- Lightweight Filter
- ↓
- Message Classifier
- ↓
- Important Messages Only → AI Processing
- ↓
- Database Storage
- ↓
- Daily / Weekly Reports
- This pipeline can reduce AI usage by up to 90% by ignoring irrelevant messages early.

Background Processing Strategy

- Heavy AI analysis should run in scheduled jobs instead of real-time.
- Example schedule:
 - Every hour – analyze new messages
 - Every day – generate summary report
 - Every week – contributor report
- This keeps the bot responsive and avoids unnecessary AI workload.

Recommended Technology Stack

- Discord Bot: Python with discord.py
- AI Runtime: Ollama
- Suggested Model: Mistral 7B or similar open-source LLM
- Backend API: FastAPI
- Database: SQLite initially, upgrade to PostgreSQL
- Frontend Dashboard: React or Next.js

Final Design Trick – Modular Plugin Architecture

- To prevent the project from turning into unmanageable code, structure the bot as a modular plugin system.

- Each capability should be an independent module:
- • server_manager
- • message_listener
- • ai_analyzer
- • bug_clusterer
- • contributor_system
- • dashboard_api
- The core bot only handles events and dispatches them to modules.
- Example flow:
- Discord Event → Event Bus → Module Handlers
- Advantages:
- • easy to add new features
- • modules can be turned on/off
- • easier debugging
- • easier future productization
- • allows the system to grow without rewriting the core bot

High-Level System Architecture

- Discord Server
- ↓
- Bot Core (Event Dispatcher)
- ↓
- Modules
- • Server Manager
- • Message Listener
- • AI Analyzer
- • Bug Intelligence
- • Contributor System
- ↓
- Database
- ↓
- AI Processing (Ollama)
- ↓
- Dashboard API
- ↓
- Frontend Dashboard

Long-Term Vision

- When fully developed, the system becomes a community intelligence platform that:
- • runs the Discord server
- • organizes feedback automatically
- • detects and prioritizes bugs
- • tracks feature requests
- • identifies valuable contributors
- • produces development insights for the Vellum platform